

Atividade 05 – Interface

→ Este exercício trata-se de uma evolução da Atividade 04.

Embora o conceito de Herança Múltipla, isto é, uma classe (filha) ser criada com base em mais de uma classe base (mãe), vide Figura 1, seja possível e comum em projetos, é importante ressaltar que existem tecnologias que não comportam este tipo de implementação.

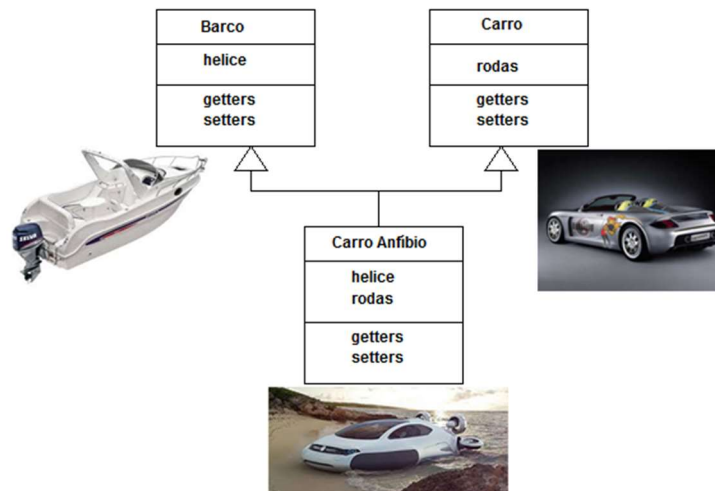
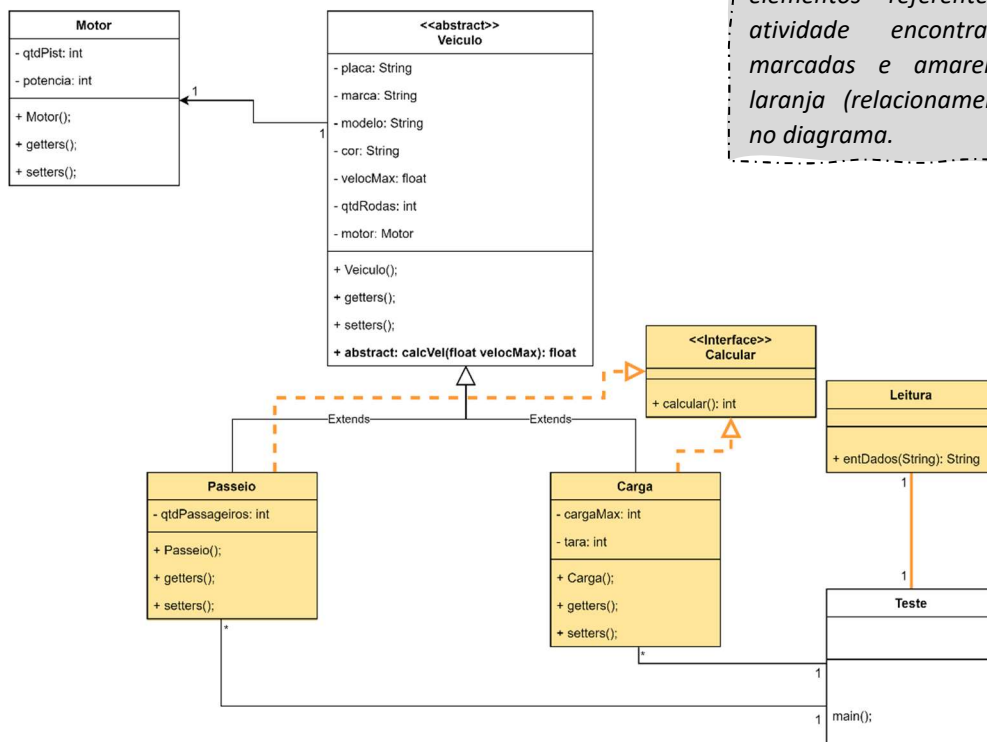


Figura 1: Projeto de Herança Múltipla (não suportado pela tecnologia Java)

Entre estas tecnologias se encontra a Java que, para resolver esta situação, desenvolveu uma estrutura de dados denominada Interface.

1) OBSERVE O SEGUINTE DIAGRAMA DE CLASSES



Obs.: As alterações e novos elementos referentes à atividade encontram-se marcadas e amarelo e laranja (relacionamentos) no diagrama.

- 2) UTILIZE O CÓDIGO DESENVOLVIDO NA ATIVIDADE 04 E DESENVOLVA OS NOVOS ELEMENTOS APRESENTADOS NO DIGRAMA ACIMA. ABAIXO, SEQUE A LISTA DE REQUISITOS A SEREM SEGUIDOS:

NÃO DEVERÁ SER UTILIZADO O RECURSO DE GUI (INTERFACE GRÁFICA)

A classe **Teste** deverá conter o **Menu Inicial** com as seguintes opções (e suas funcionalidade desenvolvidas):

```
Sistema de Gestão de Veículos - Menu Inicial

1. Cadastrar Veículo de Passeio
2. Cadastrar Veículo de Carga
3. Imprimir Todos os Veículos de Passeio
4. Imprimir Todos os Veículos de Carga
5. Imprimir Veículo de Passeio pela Placa
6. Imprimir Veículo de Carga pela Placa
7. Sair do Sistema
```

Observações sobre as opções do Menu Inicial:

- **Opções 5 e 6:** O usuário informa a placa do veículo e o programa deve executar uma busca no vetor do respectivo tipo de veículo e imprimir os dados deste;
- **Opção 7:** o sistema será encerrado apenas quando o usuário escolher esta opção.

- 3) O SISTEMA DEVERÁ PERMITIR QUE:

- a. A cada operação realizada no sistema, este retornará ao Menu Inicial;
- b. Armazenar 5 de veículos de cada tipo. Crie 2 **VETORES**, um para os veículos de passeio e outro para os de carga;
- c. Ao inserir um novo veículo o sistema deverá verificar se já existe um veículo cadastrado com a mesma placa independente do tipo de veículo, ou seja, deve-se verificar nos dois vetores. Caso exista, deverá informar ao usuário sobre esta existência e deverá voltar ao Menu Inicial.
- d. Ao final do cadastro de cada veículo, deverá ser perguntado se deseja cadastrar mais um (do mesmo tipo), caso a resposta seja **NÃO** deverá voltar ao Menu Inicial;
- e. A velocidade de cada veículo sempre será armazenada em Km/h, porém o método abstrato `calcVel` (sobrescrito nas classes `Passeio` e `Carga`) deverá calcular (não alterando o valor do atributo `velocMax`) e retornar o resultado da seguinte forma:
 - i. Se o veículo for de passeio, deverá converter de Km/h para M/h (metros por hora) e retornar este valor;

1 kilometer/hour = 1000 meter/hour

- ii. Caso seja de carga, converterá de Km/h para Cm/h (centímetros por hora) e retornar este valor;

1 kilometer/hour = 100000 centimeter/hour

Use o método `calcVel(float velocMax)`, da classe-mãe, para fazer este cálculo.

Atenção:

- O método `calcVel(float velocMax)` **NÃO** deve alterar o valor do atributo `velocMax`, **apenas convertê-lo** e retornar o valor convertido para que seja exibido na tela por meio da classe Teste;

- f. Sempre que se optar pela opção 5 ou pela 6 do Menu Inicial, estes valores serão apresentados junto com as demais informações do respectivo veículo;
- g. O método “calcular” da interface “Calc”, quando sobrescrito:
- i. Na classe Passeio: retornará a soma das quantidades de letras existentes em todos os atributos do tipo String;
 - ii. Na classe Carga: retornará a soma de todos os valores contidos nos atributos numéricos.
- h. Perceba que, pelo diagrama não há uma associação entre as classes Veículo e Teste nem entre Motor e Teste. Não poderá ser declarado um objeto do tipo Motor nas classes: Passeio, Carga e Teste. Somente haverá declaração na classe-mãe Veiculo;
- i. Todas entradas e saídas de dados deverão ser feitas na classe Teste:
- i. Entrada de dados: **utilize o método entDados da classe Leitura**;
 - ii. Saídas: como achar melhor, porém **deve imprimir os dados na mesma sequência dos atributos da classe-mãe Veiculo** e depois destes os resultados dos métodos “calcular” e “calcVel”;

4) RESTRIÇÕES

- a. NÃO PERMITA QUE:
- i. Que seja possível instanciar objetos diretos da classe mãe (Veiculo)
 - ii. Sobrescrevam os métodos setters da classe Veiculo;
 - iii. Sobrescrevam os métodos das classes filhas (Passeio e Carga);
 - iv. Que as classes filhas (Passeio e Carga) sejam herdadas;

5) O QUE SERÁ AVALIADO

- a. Construção das classes, com os atributos e métodos conforme descritos no diagrama de classe do item 01, não podendo fazer implementações diferentes do diagrama de classes.
- b. Relacionamento de herança entre as classes.
- c. Validação do menu, suas opções e restrições, conforme solicitado nos itens 2 e 3.
- d. Validação dos itens solicitados nos itens 2, 3 e 4.

Importante!

- Atenha-se aos nomes dos elementos (classes, atributos e métodos) conforme apresentados no diagrama, esses devem ser idênticos ao diagrama.
- Novos métodos poderão ser criados, caso julgue necessário.
- Os itens avaliados são os solicitados no enunciado. Elementos extras NÃO renderão pontos a mais.
- O não cumprimento do que foi solicitado acarretará no decréscimo da nota de acordo com a gravidade da falta.
- A justificativa para qualquer desconto será colocada, pelo avaliador, no campo de feedback de cada Atividade.